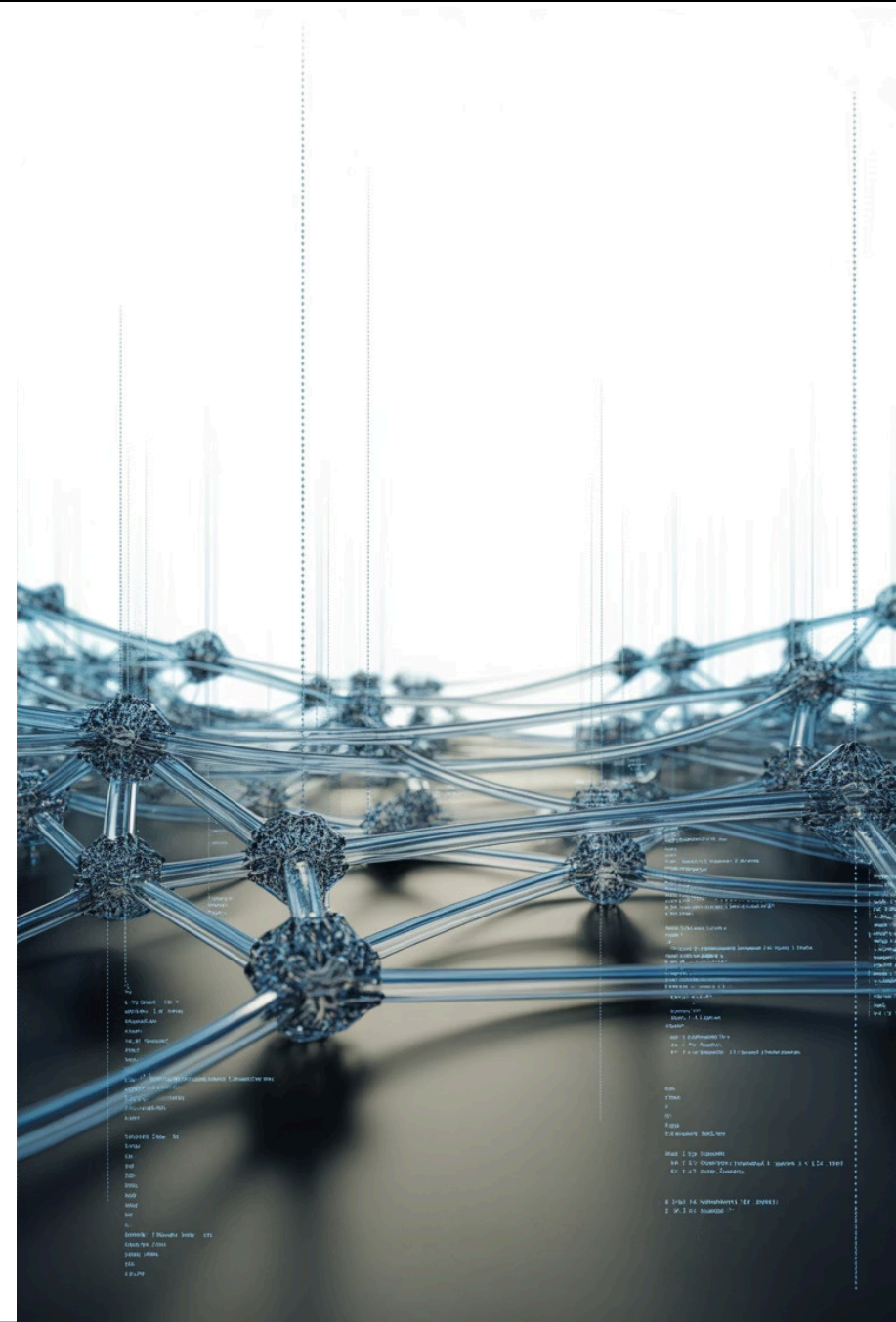


Module 5

Advanced Machine Learning Techniques

Presentation prepared with AI assistance



Introduction: AI Revolution and Python

Artificial intelligence is transforming the way we solve complex analytical problems. Python has become the dominant programming language in this field, offering an ecosystem of tools that enable the construction of advanced machine learning systems.

Modern AI applications include image recognition, natural language processing, recommendation systems, and autonomous vehicles.



Modeling: Key to Success

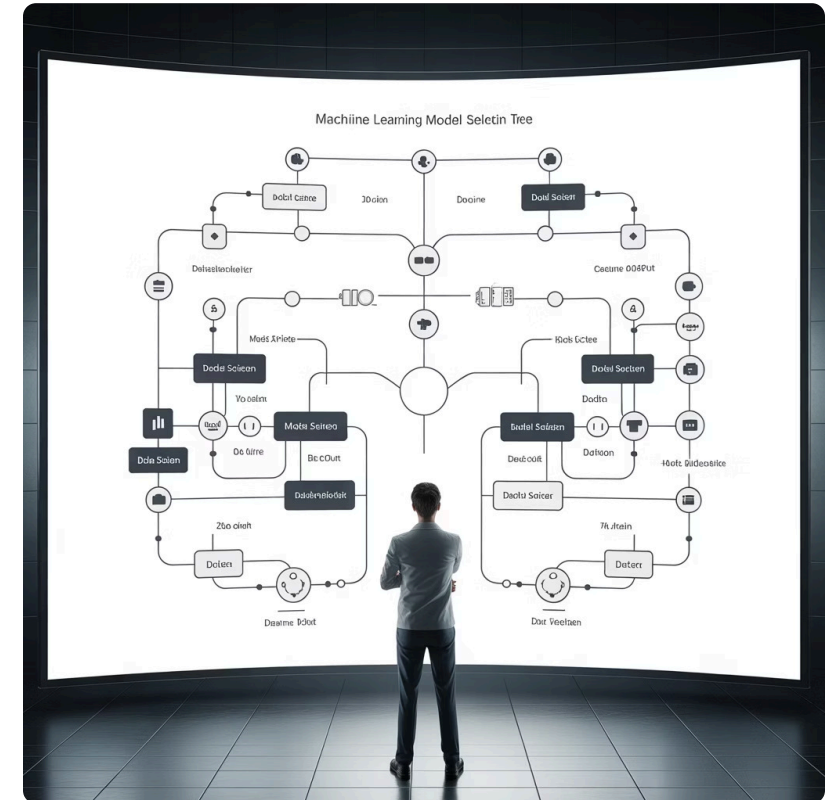
Choosing the Right Model

The decision regarding the model type depends on the nature of the problem:

- **Classification:** assigning objects to categories
- **Regression:** predicting continuous values
- **Clustering:** grouping similar observations

The Importance of Optimization

Model optimization leads to improved predictive performance, reduced training time, and increased accuracy on new data.



Model Optimization: First Steps



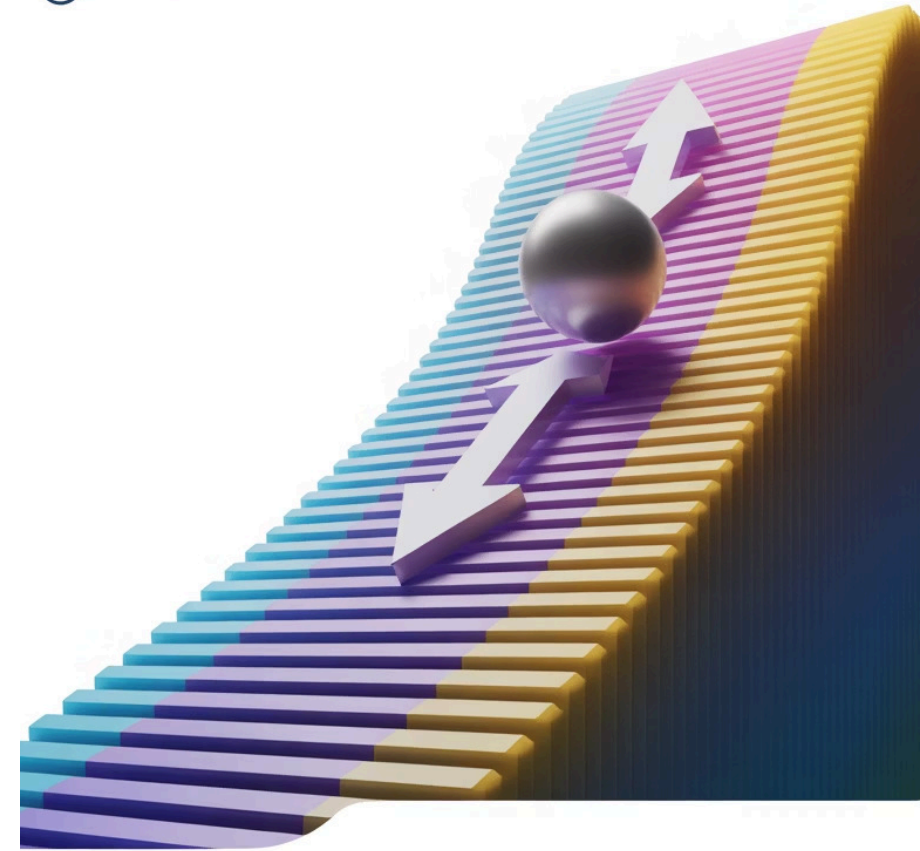
Cost Function

A mathematical measure of model errors – it quantifies the difference between predictions and actual values



Optimization Methods

Gradient Descent, Adam, SGD – algorithms that iteratively minimize the cost function by updating model parameters



Process

Gradient Descent Optimization

Gradient Descent: Step by Step


$$\frac{f}{dx}$$

Gradient Calculation

The gradient indicates the direction of the steepest ascent of the cost function. We calculate partial derivatives with respect to all model parameters.



Weight Update

Parameters are updated in the opposite direction of the gradient, which leads to the minimization of the cost function according to the formula: $\theta = \theta - \alpha \nabla J(\theta)$



Iteration

The process is repeated multiple times until convergence is achieved or a specified number of training epochs.

Adam: Adaptive Optimization



Combination of **Momentum** and **RMSprop**

The Adam algorithm (Adaptive Moment Estimation) combines the advantages of two optimization methods:

- **Momentum**: accelerates learning in consistent directions
- **RMSprop**: adaptive adjustment of the learning rate

Efficiency

Fast convergence due to individual learning rates for each parameter. An excellent choice for deep learning models.

Hyperparameter Tuning: The Art of Precision

What are hyperparameters?

Model settings determined before training begins, such as the learning rate, the number of layers in a neural network, batch size, or activation function.

Significance in the learning process

Hyperparameters have a crucial impact on the model's ability to learn and generalize. Their appropriate selection can significantly improve prediction results.

Difference from parameters

Unlike parameters (weights), which are learned during training, hyperparameters must be specified beforehand and require systematic tuning.

Hyperparameter Tuning Methods



Grid Search

Grid search – systematic testing of all possible combinations of specified hyperparameters within a given range



Randomized Search

Random sampling from the hyperparameter space – more efficient for high-dimensional search spaces



Bayesian Optimization

Intelligent search utilizing a probabilistic model to predict promising regions of the hyperparameter space



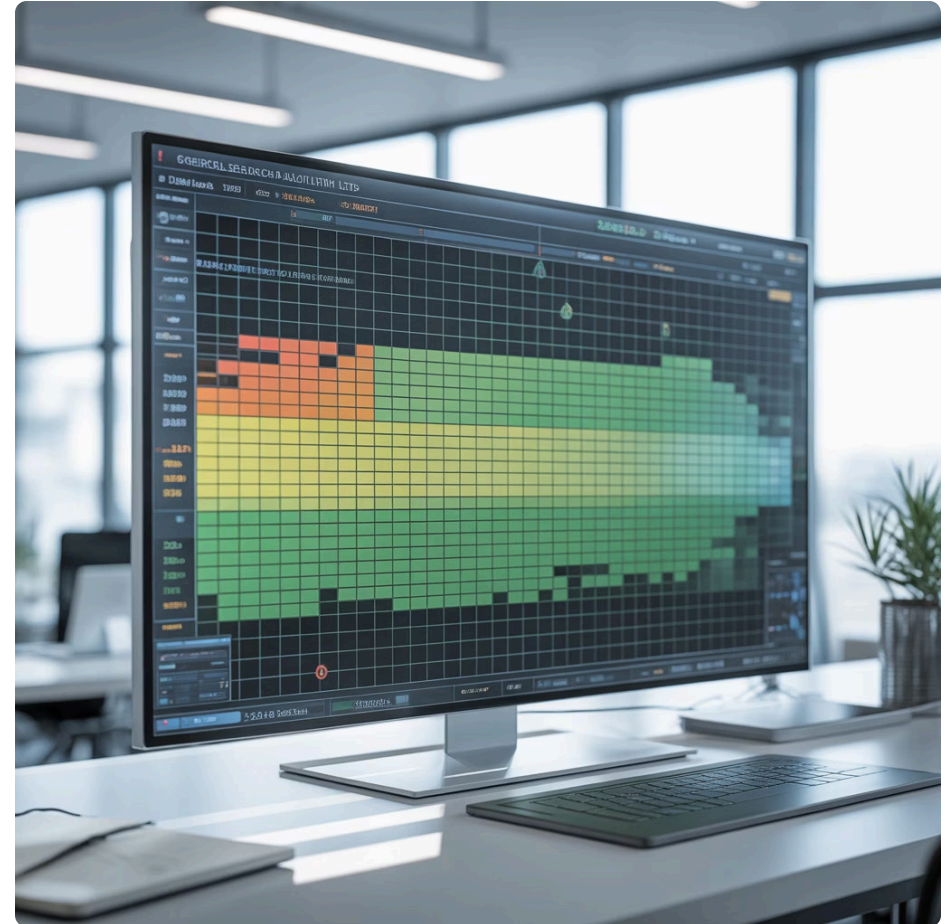
Grid Search: A Systematic Approach

Advantages

- Simple and easy to implement
- Guaranteed to find the optimal combination within the defined grid
- Deterministic results
- Easy interpretation of results

Disadvantages

- Time-consuming, especially for large parameter spaces
- Complexity grows exponentially with the number of hyperparameters
- Ineffective for continuous spaces





Randomized Search: A Chance for Discovery

Advantages of the Method

Significantly faster exploration of hyperparameter space, ability to explore a wider range of values, better results with a limited time budget

Disadvantages of the Method

Less certainty of finding globally optimal settings, results may vary between runs, requires a greater number of iterations for certainty

Research shows that Randomized Search often achieves comparable results to Grid Search in a fraction of the computation time.

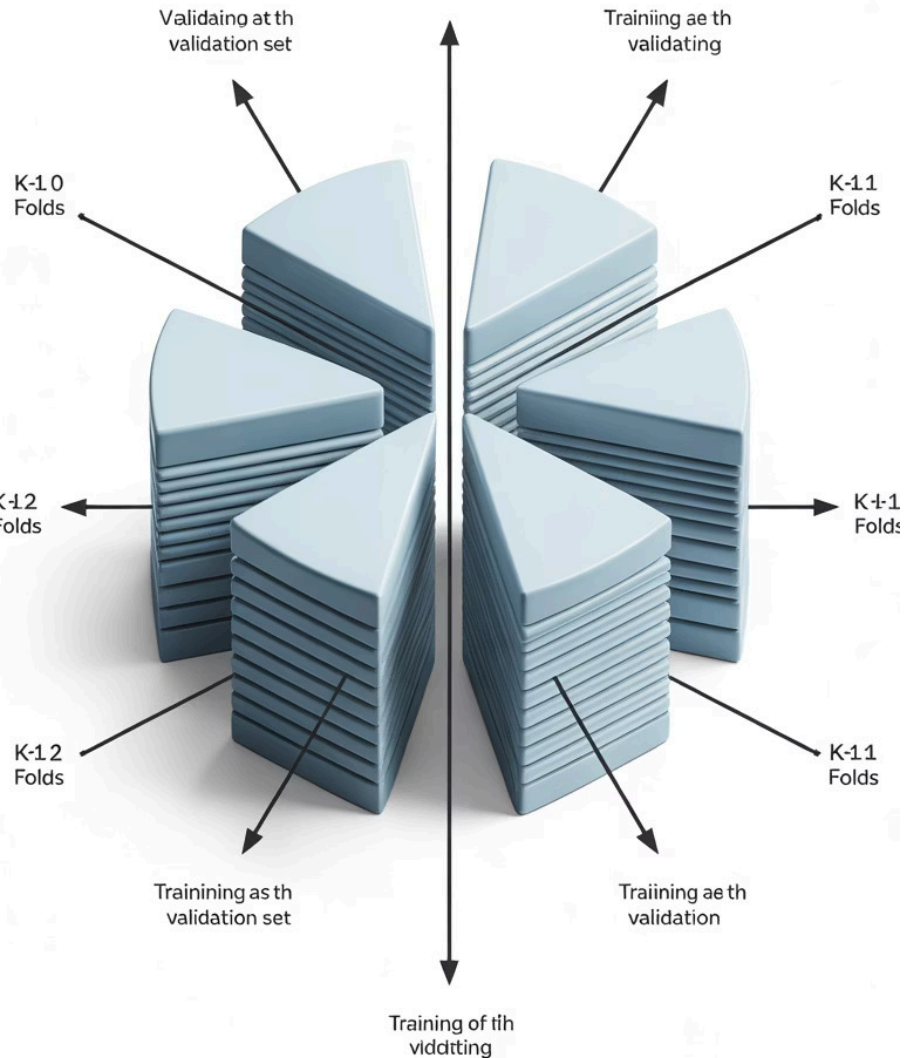
Bayesian Optimization: Intelligence in Action



Efficiency: Rapid convergence to optimal values with a minimal number of evaluations, especially effective for costly objective functions.

Cross-Validation

K-Fold Diagram



Cross-Validation: Robust Evaluation

01

Splitting training data

Data divided into k equal parts (folds), where k typically equals 5 or 10

02

Iterative training

Model trained k times, each time using a different fold as the validation set

03

Result aggregation

The average of k results provides a more reliable assessment of model performance

Significance: Cross-validation reduces the risk of overfitting and provides a more realistic assessment of the model's generalization ability on new, unseen data.

Overfitting: The Model Trap

Problem Characteristics

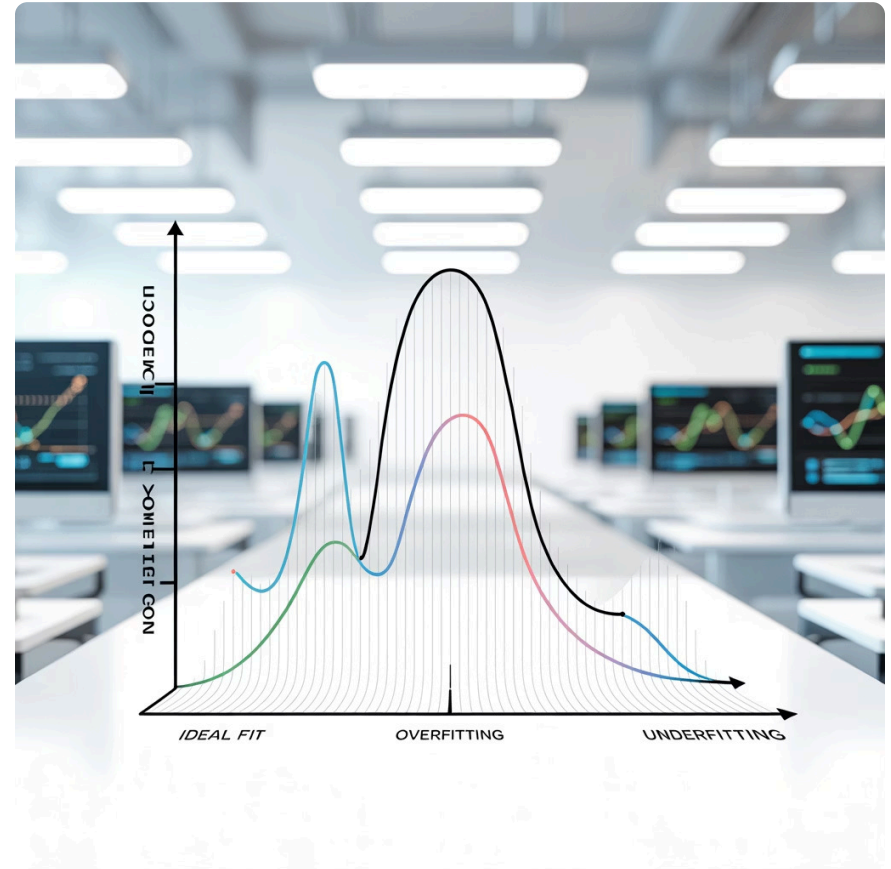
A model that is too well fitted to the training data learns not only patterns, but also noise and anomalies specific to the training set.

Consequences

- Excellent accuracy on training data
- Poor generalization to new data
- Model fails to capture true patterns
- High prediction variance

Recognition

A large difference between metrics on the training set and the validation set signals overfitting.



Regularization: Defense Against Overfitting

1

L1 Regularization (Lasso)

Adds a penalty proportional to the sum of the absolute values of the weights. Leads to sparse solutions (many weights = 0), which enables feature selection.

2

L2 Regularization (Ridge)

Adds a penalty proportional to the sum of the squares of the weights. Reduces the values of all weights, decreasing the model's sensitivity to individual features.

3

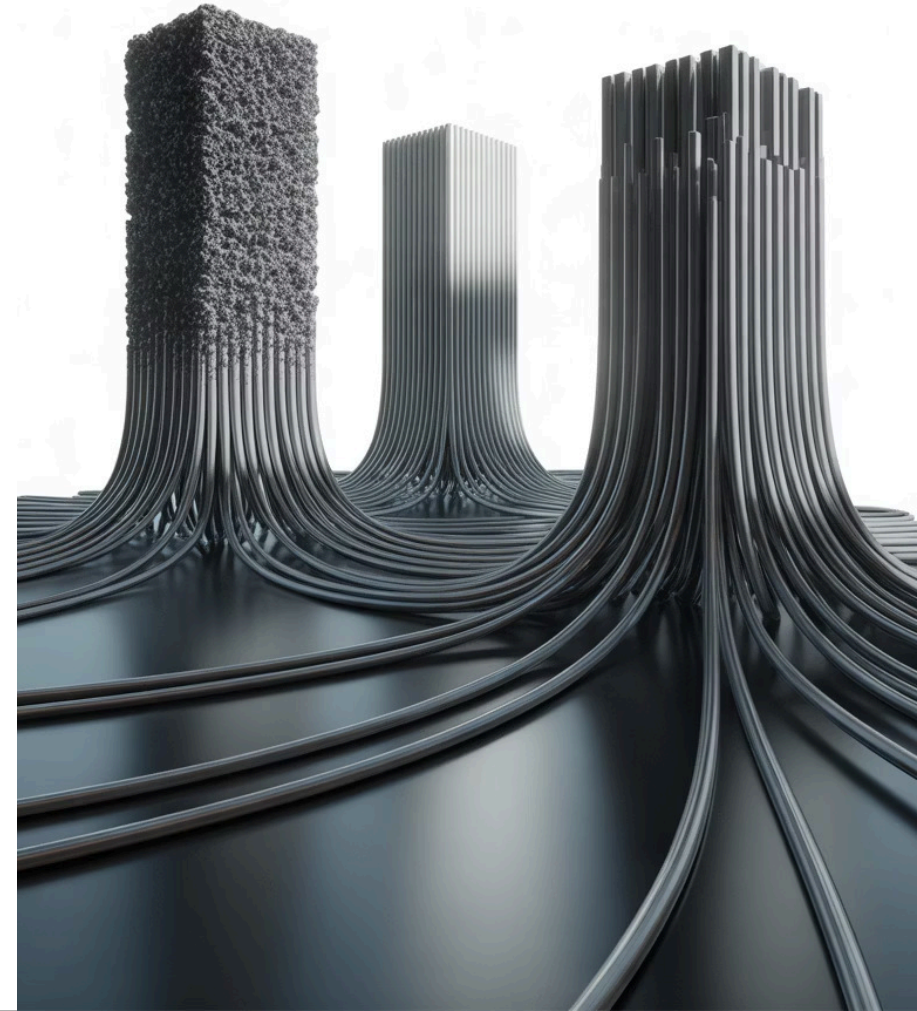
Dropout

Randomly deactivating neurons during neural network training. Prevents co-adaptation of neurons and increases model robustness.

L1
Regularization
(Lasso)

L2
Ridge
(Ridge)

Elastic
Regularization
Elastic Net



L1 and L2 Regularization: Reducing Complexity

L1 Regularization

Cost function: $J(\theta) = \text{MSE} + \lambda \sum |\theta_i|$

Application: Feature selection by eliminating irrelevant variables

- Creates sparse models
- Automatic feature selection
- Interpretable models



L2 Regularization

Cost function: $J(\theta) = \text{MSE} + \lambda \sum \theta_i^2$

Application: Reducing the magnitude of weights for all parameters

- Stable numerical solutions
- Variance reduction
- Uniform weight reduction



Dropout: Learning Robustness

1

Training Phase

Randomly deactivating neurons with a specified probability (typically 0.2-0.5) in each iteration

2

Regularization Effect

The network learns redundant representations, preventing co-adaptation of neurons and forcing it to learn more general patterns

3

Testing Phase

All neurons active, weights scaled by dropout probability, creating an ensemble effect

Dropout significantly increases model robustness against overfitting, especially in deep neural networks with many parameters.



Practical Tools: Scikit-learn, Keras

GridSearchCV

Built-in implementation of grid search with cross-validation, automatic testing of parameter combinations

RandomizedSearchCV

Random search of hyperparameter space with flexible sampling distributions

Keras Tuner

Advanced tuning for Keras models, supports Hyperband, Bayesian Optimization, and Random Search

These libraries provide complete solutions for automating the optimization process, significantly accelerating the development of machine learning models.

Example: Model Tuning

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

# Definicja siatki hiperparametrów
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Inicjalizacja modelu
rf = RandomForestClassifier(random_state=42)

# Grid Search z 5-krotną walidacją krzyżową
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    n_jobs=-1,
    verbose=2,
    scoring='accuracy'
)

# Trening i optymalizacja
grid_search.fit(X_train, y_train)

# Najlepsze parametry
print("Optymalne hiperparametry:", grid_search.best_params_)
print("Najlepsza dokładność CV:", grid_search.best_score_)
```

The code above demonstrates the practical application of GridSearchCV for optimizing a Random Forest classifier, systematically searching the hyperparameter space.